Toronto Metropolitan University

Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title	System-on-Chip Design
Course Number	COE 838
Semester	Winter 2026

Instructor:	Dr. Gul Khan
Teaching Assistants (TAs):	Mr. Joshua Fernandes

Assignment Type	Lab 3
Assignment Topic	DE1-SoC Tutorial

Submission Date	March 6, 2026
Due Date	March 4, 2026 5:00 PM

Last Name	First Name	Student Number	Section	Signature
Floreza	Dexter Ryan	500946679	02	D.R.F.

Introduction

Modern embedded systems rely on hardware-software co-design, where programmable logic and processor subsystems operate together within a single System-on-Chip (SoC). The DE1-SoC platform features a Cyclone V FPGA fabric tightly integrated with a dual-core ARM Cortex-A9 Hard Processor System (HPS), providing an appropriate environment to explore this interaction in a practical setting.

The objective of this lab is to develop and integrate a custom FPGA hardware design and embedded software running on the HPS. Using Quartus and Platform Designer (Qsys), a hardware system is constructed such that it interacts with LEDs and seven-segment displays through memory-mapped registers accessible to the processor. The FPGA design is synthesized, compiled, and programmed.

Methodology

This laboratory implements the HPS/FPGA design flow in SoC Design using a combination of Quartus II, Qsys, and VHDL. The first part of this system revolves around using Quartus to configure the Project Directory, selecting the family, device, and correct packages. Overall, this laboratory uses the Cyclone V Device, the FBGA package, and the 5CSEMAF531C6 device. The objective was to configure the FPGA hardware components, compile the design, and develop embedded software capable of controlling hardware peripherals such as LEDs and seven-segment displays from the HPS.

The hardware system was first designed using Intel Quartus and Platform Designer. A system architecture was constructed that connected FPGA peripherals to the HPS through the lightweight HPS-to-FPGA AXI bridge. This bridge enables the processor to access FPGA components as memory-mapped devices. Peripherals such as LEDs, switches, and seven-segment displays were added to the system and assigned specific base addresses within the memory map. Once the hardware system was configured, the design was synthesized and compiled in Quartus to generate the FPGA configuration file, which was then programmed onto the DE1-SoC board.

After completing the hardware configuration, embedded software was developed to control the FPGA peripherals from the HPS. The software was written in the C programming language using the Intel SoC Embedded Development Suite. The program mapped the lightweight HPS-to-FPGA bridge address space into user space using memory-mapping functions, allowing the software to directly access peripheral registers. Pointer variables were then used to read from or write to the memory addresses corresponding to the LEDs and seven-segment displays.

Once compiled, the application was transferred to the DE1-SoC system and executed through a serial terminal environment. The terminal interface allowed interaction with the program and

enabled the observation of system messages during execution. When the program ran, control signals were written to the mapped hardware registers, resulting in visible changes on the board such as LED patterns and updates to the seven-segment displays.

Finally, system functionality was verified by observing the hardware outputs and confirming that the software successfully controlled the FPGA peripherals through the HPS–FPGA interface. The correct operation of the LEDs and seven-segment displays demonstrated successful integration of the FPGA hardware design, the memory-mapped communication interface, and the embedded software running on the processor.

Results

The results of the laboratory demonstrate the successful integration of the FPGA hardware design with the embedded software running on the Hard Processor System (HPS). When the system was executed on the DE1-SoC platform, the processor was able to communicate with FPGA peripherals through the lightweight HPS-to-FPGA AXI bridge using memory-mapped registers. As the program ran, visible hardware outputs were observed on the board, including animated LED patterns, dynamic updates on the seven-segment displays, and real-time system messages displayed through the serial terminal interface. The terminal output provided confirmation of program execution and system state changes, while the hardware outputs verified correct communication between the processor and FPGA peripherals. Screenshots of the terminal output, Qsys system configuration, and memory-mapping configuration are presented below to illustrate the successful operation of the implemented system.

Figure 1: The Serial Terminal Window during an execution of the SoC System. The terminal updates to show the state of the system.

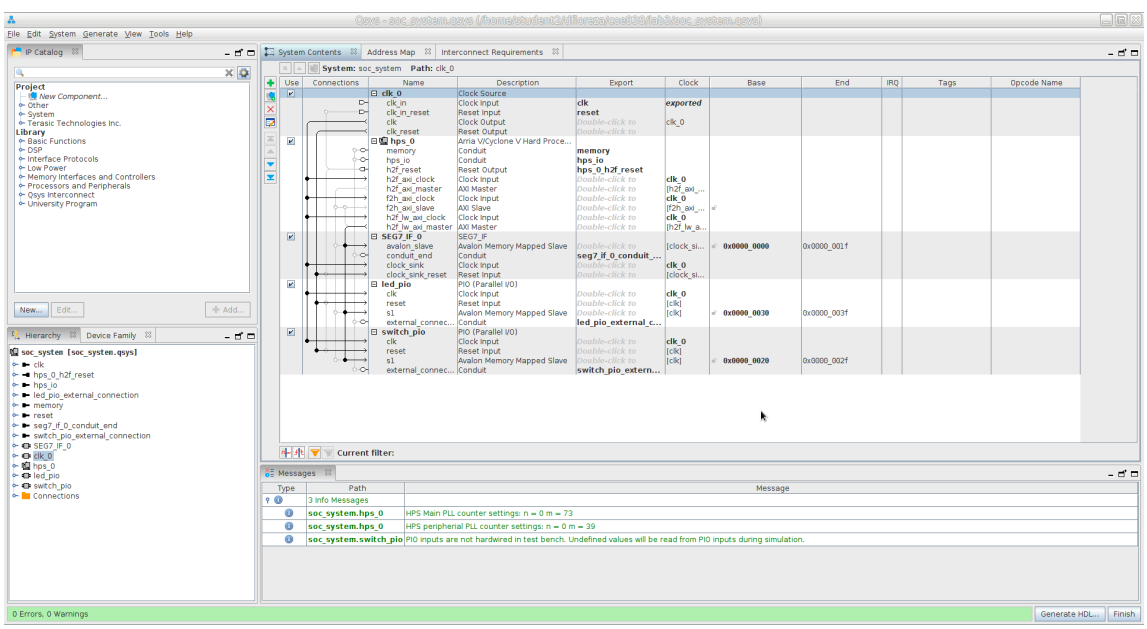

Figure 2: The QSys Window, displaying the system contents.

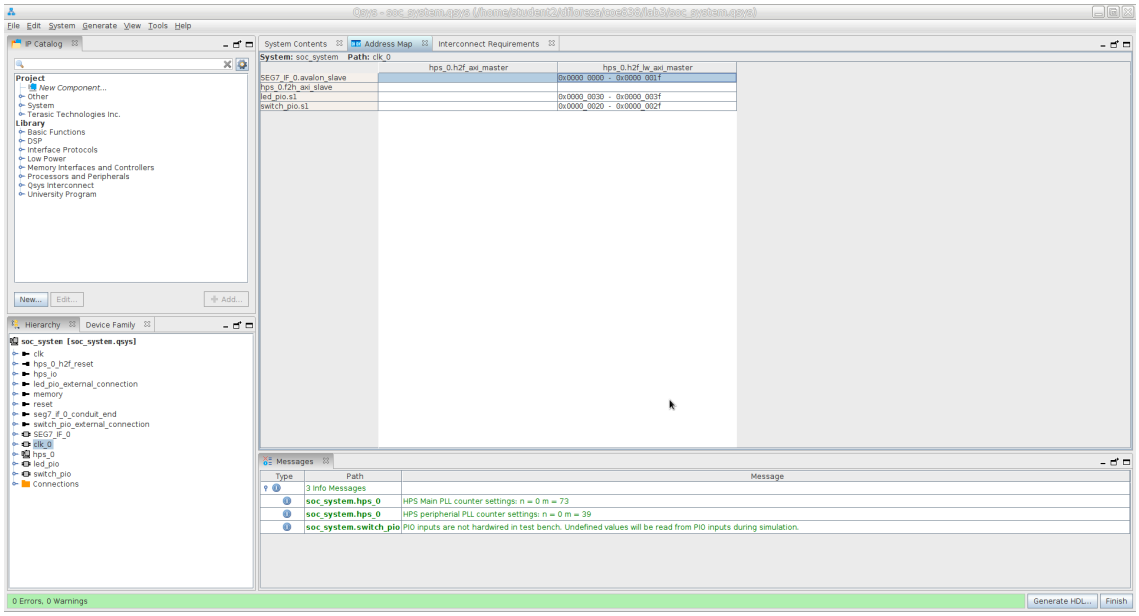

Figure 3: The QSys Window displaying the mapping

Challenges

Some challenges encountered during this laboratory included understanding and managing the FPGA/SoC design flow. The System-on-Chip development process involves multiple stages, including hardware design in Quartus, system integration through Platform Designer (Qsys), generation of hardware description files, compilation and synthesis of the FPGA configuration, and the development of embedded software that interfaces with the hardware. Because these stages are tightly interconnected, errors occurring in one stage often propagated into later stages of the workflow. As a result, becoming familiar with the order of operations in the design flow proved essential for resolving bugs, debugging errors, and troubleshooting system behaviour.

A significant source of error during the lab arose from Quartus retaining previously generated design artifacts and producing outdated header (.h) files and hardware description outputs. These header files are automatically generated from the system configuration and define the memory-mapped base addresses used by the software to communicate with FPGA peripherals. When modifications were made to the hardware design, such as adding switches, Quartus and Qsys would sometimes continue referencing older compiled files rather than regenerating updated configuration data. Consequently, when commands were executed through the NIOS or embedded development environment to regenerate software support files, the produced header files did not reflect the most recent hardware configuration.

This discrepancy created inconsistencies between the hardware system and the embedded software. The software attempted to access memory addresses corresponding to peripherals defined in outdated header files, which resulted in incorrect behaviour or non-functional outputs on the board. For example, switches or LEDs that had been newly connected within the hardware system would not respond correctly because the software was referencing obsolete address definitions. Resolving this issue required manually cleaning the project directory, deleting old build artifacts, and regenerating the system files so that Quartus could produce updated header files consistent with the most recent hardware configuration. This additional troubleshooting significantly increased the time required to complete the lab and highlighted the importance of understanding how Quartus manages generated system files and compilation artifacts.

Conclusion

This laboratory demonstrated the practical implementation of a System on Chip architecture by integrating FPGA hardware with embedded software running on the ARM Cortex A9 Hard Processor System of the DE1 SoC platform. Through the use of Quartus, a hardware system was constructed that allowed FPGA peripherals such as LEDs and seven segment displays to be accessed through memory mapped registers. The embedded software developed in C was able to communicate with these peripherals through the lightweight HPS to FPGA bridge, successfully producing visible LED patterns, updates on the seven segment displays, and system messages through the serial terminal interface. These results confirmed the correct operation of the hardware design and its interaction with the processor.

The laboratory also provided valuable insight into the complete SoC development workflow, including hardware design, system integration, synthesis, compilation, and embedded software development. Challenges related to outdated build artifacts and incorrect header file generation emphasized the importance of properly managing project files and ensuring that the hardware configuration and software environment remain synchronized. Overall, the experiment demonstrated how programmable logic and processor based software can operate together within an integrated system, providing practical experience with embedded system design and establishing a foundation for more advanced System on Chip development.

Appendix

The code has been attached separately.

A video of the SoC demonstration can be found on this link:
https://drive.google.com/file/d/18_HiTohm4m10vXItO44ZEy9HtVhn1zn1/view?usp=sharing

References

[1] G. Khan, “DE1-SoC Tutorial,” COE838: System-on-Chip Design, Lab 3, Dept. of Electrical, Computer, and Biomedical Engineering, Toronto Metropolitan University, Winter 2026

[2] G. Khan “DE1-SoC Labs Introduction to HPS/FPGA Systems,” Dept. of Electrical, Computer, and Biomedical Engineering, Toronto Metropolitan University, Winter 2026.